

Traceveil: AI-Assisted Real-Time Smart Home IoT Digital Forensics and Incident Response Platform

INTERIM PROJECT REPORT

Submitted in partial fulfilment of the Capstone Project requirements

Program: Btech

Stream / Department: Cybersecurity

by

Ayra Jha (K028) | SAP ID: 70102300052

Atharva Rangnekar (K053) | SAP ID: 70102300014

Aarya Rao (K054) | SAP ID: 70102300046

Under the supervision of

Prof. Rohit Suryawanshi

[Cybersecurity, MPSTME]

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT &

ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

Academic Year: [2026/27]

CERTIFICATE

This is to certify that the project entitled “Traceveil: AI-Assisted Real-Time Smart Home IoT Digital Forensics and Incident Response Platform” has been carried out by Ayra Jha (K028), Atharva Rangnekar (K053), and Aarya Rao (K054) under the guidance of Prof. Rohit Suryawanshi and is submitted as an interim Capstone Project report to MPSTME, SVKM’s NMIMS (Deemed-to-be University), Mumbai.

The report covers the project background, literature survey, methodology, system design, and implementation status up to the midterm review.

Project Mentor

Prof. Rohit Suryawanshi

Examiner / Panel Member

Date: _____

Place: Mumbai

Head of Department

ACKNOWLEDGEMENT

We sincerely thank our faculty mentor, Prof. Rohit Suryawanshi, for his guidance and feedback throughout Traceveil. We also thank the MPSTME faculty and review panel, and acknowledge the researchers and organizations whose datasets, standards, and publications informed the project. We thank all team members for their contributions to design, implementation, testing, integration, and documentation.

ABSTRACT

Smart-home environments generate heterogeneous evidence across sensors, device telemetry, network records, application logs, and vendor-specific ecosystems. During an incident, differences in timestamps, identifiers, formats, and event semantics make reconstruction difficult. Existing research provides useful acquisition frameworks, security datasets, and AI-assisted techniques, but investigator-facing normalization and reproducible cross-source analysis remain practical challenges.

Traceveil is an academic smart-home IoT Digital Forensics and Incident Response (DFIR) prototype that unifies historical/batch evidence and controlled live IoT telemetry. Both input paths are validated and normalized into a versioned Canonical Event Model that preserves evidence identity, source hashes, row or sequence references, device/entity information, and normalization provenance. A deterministic Python engine then applies versioned rules and behavioural baselines, calculates bounded risk scores, correlates related events, reconstructs timelines, and generates graph and chart-ready investigation outputs.

The platform uses FastAPI and Pydantic for backend contracts, SQLite for local persistence, Pandas and NetworkX for analytical outputs, and a React/Vite dashboard for case-centric investigation. The live path uses authenticated telemetry intake and a transport-neutral contract that supports controlled hardware integration. Qwen 9B through Ollama is treated only as a bounded explanatory layer: it may summarize persisted forensic facts, but Python remains the forensic authority and the model cannot calculate or alter findings.

The team completion audit dated 6 September 2026 records 93 of 143 planned roadmap sub-items as implemented and verified, giving 65.0% formal roadmap completion. Core foundation, evidence validation, canonical normalization, deterministic analytics, investigation APIs, and Live IoT Integration are verified for the interim review. Remaining formal work centers on final acceptance of real-time delivery, AI/reporting workflows, and the complete physical-device demonstration.

TABLE OF CONTENTS

List of Figures.....	ii
List of Tables	ii
Abbreviations	ii
Chapter 1 Introduction	1
1.1 Background of the Project Topic	1
1.2 Motivation and Scope of the Report	1
1.3 Problem Statement	2
1.4 Salient Contribution	2
1.5 Organization of the Report.....	2
Chapter 2 Literature Survey	3
2.1 Introduction to the Overall Topic	3
2.2 Detailed Literature Survey	3
2.3 Comparative Positioning and Research Gap.....	4
2.4 Standards and Design Implications	5
Chapter 3 Methodology and Implementation	6
3.1 Methodology Overview and Block Diagram	6
3.2 Hardware Description	7
3.3 Software Description.....	8
3.4 Flowchart and Core Algorithms.....	8
3.5 Current Implementation Status and Future Extensions	9
References	11

LIST OF FIGURES

Figure 3.1 Traceveil logical architecture	6
Figure 3.2 Evidence import and deterministic analysis activity model	8

LIST OF TABLES

Table 2.1 Literature and standards reviewed	3
Table 3.1 Selected hardware components.....	7
Table 3.2 Software technology stack	8
Table 3.3 Deterministic detection rules	9

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CASAS	Center for Advanced Studies in Adaptive Systems smart-home dataset project
DFIR	Digital Forensics and Incident Response
HMAC	Hash-based Message Authentication Code
IoT	Internet of Things
LLM	Large Language Model
MQTT	Message Queuing Telemetry Transport
NIST	National Institute of Standards and Technology
NTP	Network Time Protocol
SIEM	Security Information and Event Management
SSE	Server-Sent Events
TLS	Transport Layer Security

Chapter 1

Introduction

Smart-home incidents can generate evidence across sensors, hubs, applications, network records, and cloud-linked devices, but that evidence is fragmented by vendor formats, identifiers, and timestamps. Traceveil is a local academic DFIR prototype that validates and normalizes selected batch evidence and controlled live telemetry into one provenance-preserving representation before any forensic finding is calculated.

1.1 Background of the Project Topic

NIST SP 800-86 frames forensic work around Collection, Examination, Analysis, and Reporting [13], while SP 800-92 emphasizes structured log handling [14]. Smart-home literature shows that proprietary formats, inconsistent clocks, and device-specific acquisition methods make cross-device investigation difficult [5]-[10].

Traceveil evaluates this problem using TON_IoT [1] and CASAS [2] as primary public sources, with CICIoT2023 [3] and IoT-CAD [4] providing additional context. The project focuses on validation, normalization, deterministic analysis, correlation, timelines, and investigator-facing review rather than training a new attack classifier.

1.2 Motivation and Scope of the Report

The project is motivated by the need to combine historical evidence and controlled live IoT telemetry in a reproducible investigator workflow. Python remains the analytical authority; the local Qwen/Ollama assistant is limited to evidence-grounded explanation of persisted results.

The interim scope includes evidence validation and hashing, Canonical Event v1.0, deterministic findings/risk/correlation/timelines, case APIs, React investigation views, authenticated laboratory telemetry, replayable delivery, and bounded local assistant/report workflows. Production-scale fleet monitoring, automatic containment, enterprise SIEM replacement, and legal-admissibility claims are outside scope.

1.3 Problem Statement

Smart-home IoT investigations are hindered by heterogeneous formats, timestamps, identities, and the separation of historical evidence from live telemetry. Traceveil investigates how both can be mapped to one provenance-preserving event model and analysed reproducibly while AI remains outside the forensic decision path.

1.4 Salient Contribution

- A versioned Canonical Event Model that unifies selected batch datasets, controlled simulation, and accepted live telemetry while preserving evidence provenance.
- A deterministic investigation engine with versioned rules, behavioural baselines, bounded five-factor risk scoring, correlation, incidents, timelines, graphs, and aggregates.
- An authenticated live path with sequence-integrity controls and replayable delivery, plus a bounded local Qwen/Ollama explanation layer and controlled ESP32 integration design.

1.5 Organization of the Report

Chapter 1 defines the problem and scope; Chapter 2 reviews prior work and the research gap; Chapter 3 presents the methodology, architecture, implementation, and interim status.

Chapter 1 Summary

Chapter 1 establishes Traceveil as a bounded academic DFIR prototype: evidence is validated and normalized first, provenance is preserved, findings are deterministic, and AI is explanatory only.

Chapter 2

Literature Survey

The literature review was used to establish the problem, select representative evidence sources, and define the project boundary. The surveyed work spans smart-home forensic frameworks, IoT security datasets, AI-assisted digital forensics, and NIST guidance.

2.1 Introduction to the Overall Topic

Recent smart-home forensic work repeatedly identifies heterogeneity as a practical obstacle: evidence may be distributed across hubs, cameras, NAS devices, mobile applications, and cloud-linked services, each with different acquisition and interpretation requirements [5]-[10]. The literature also shows a parallel growth in machine-learning and LLM-assisted forensic tools [4], [11], [12]. Traceveil uses these findings to justify a common event representation and a strict boundary in which deterministic services establish facts while the language model only explains them.

Table 2.1 Literature and standards reviewed

Refs.	Area	Relevance to Traceveil
[1]-[4]	Datasets / IoT security	Primary and secondary evidence for repeatable normalization and forensic-analysis testing.
[5]-[10]	Smart-home / IoT forensics	Heterogeneity, acquisition challenges, scalable architecture, and the research gap.
[11]-[12]	LLMs in digital forensics	Benefits of natural-language assistance and risks of hallucination/non-determinism.
[13]-[14]	NIST guidance	Collection/examination/analysis/reporting plus log-management and normalization principles.

2.2 Detailed Literature Survey

2.2.1 Smart-Home and IoT Forensic Frameworks

Shin et al. [5] validate a forensic acquisition methodology across Samsung SmartThings, Aqara, QNAP NAS, and Hikvision devices, demonstrating how proprietary formats and timestamp drift complicate investigation. The InCACCT review [6], Abdel-Fattah et al. [7], the TLDKS systematic review [8], and the smart-city survey [9] similarly describe fragmented tooling, volatile evidence, and device-specific acquisition. Kim et al. [10] move toward a generalized smart-home architecture and scalable investigation workflow.

Together, these sources support Traceveil's decision to normalize heterogeneous evidence into one investigator-facing model.

2.2.2 Benchmark Datasets

TON_IoT [1] supplies labelled IoT/IIoT telemetry and related network evidence from a realistic testbed and is one of Traceveil's primary public batch sources. CASAS [2] supplies real household sensor observations; the Milan dataset is used to exercise smart-home sensor normalization. CICIoT2023 [3] is retained as a secondary network-flow compatibility source. IoT-CAD [4] is especially relevant because it combines several forensic evidence types with AI-based attack attribution; however, its emphasis is classifier performance rather than reproducible investigator-facing correlation, timelines, and rule traces. Traceveil therefore positions itself as the deterministic investigation layer above heterogeneous evidence rather than as a new classifier.

2.2.3 AI-Assisted Digital Forensics

The LLM review in [11] and ForensicLLM [12] show that language models can assist with summarization, search, question answering, and report support, but probabilistic output creates reliability and confidentiality concerns when models are asked to establish forensic facts. These findings support two Traceveil decisions: inference is kept local in the current prototype using Qwen 9B through Ollama, and the model is forbidden from computing detections, risk scores, or correlations. It can only explain persisted records and must return references to the supplied investigation context.

2.3 Comparative Positioning and Research Gap

Commercial IoT-security and SIEM platforms provide mature monitoring and investigation capabilities, but their schemas, deployment models, and scoring logic are product-dependent. Traceveil is intentionally narrower: an academic smart-home DFIR prototype that treats heterogeneous historical evidence and controlled live telemetry as first-class inputs to the same canonical model. Among the systems reviewed for this project, we did not identify one that combines this batch/live canonical event model, deterministic forensic analysis, and a locally hosted evidence-grounded explanation layer in the same form.

The resulting research gap is therefore an integrated, reproducible workflow for cross-source IoT evidence rather than a lack of attack classifiers. This gap leads directly to the project problem statement and to the Canonical Event Model described in Chapter 3.

2.4 Standards and Design Implications

NIST SP 800-86 [13] describes a forensic process centred on Collection, Examination, Analysis, and Reporting. Traceveil follows the same evidentiary progression while adding explicit validation and canonical normalization to handle IoT heterogeneity. NIST SP 800-92 [14] informs structured log handling and consistent event representation. From the literature and standards, the project derives four design requirements: preserve provenance, distinguish observed time from ingestion time, keep analytical outputs reproducible and explainable, and treat AI narration as non-authoritative.

Chapter 2 Summary

The review confirms evidence heterogeneity as the main practical problem targeted by Traceveil and supports a provenance-preserving canonical model, deterministic investigator-facing analysis, and bounded local AI assistance. These requirements form the basis of the methodology in Chapter 3.

Chapter 3

Methodology and Implementation

Traceveil uses a staged pipeline in which evidence is admitted through defined contracts, normalized into Canonical Event v1.0, analysed by deterministic Python services, persisted in SQLite, and exposed through case-scoped APIs and the React interface. Batch and live inputs converge before analysis, so the same rules, risk model, correlation logic, and timeline generation can be reused.

3.1 Methodology Overview and Block Diagram

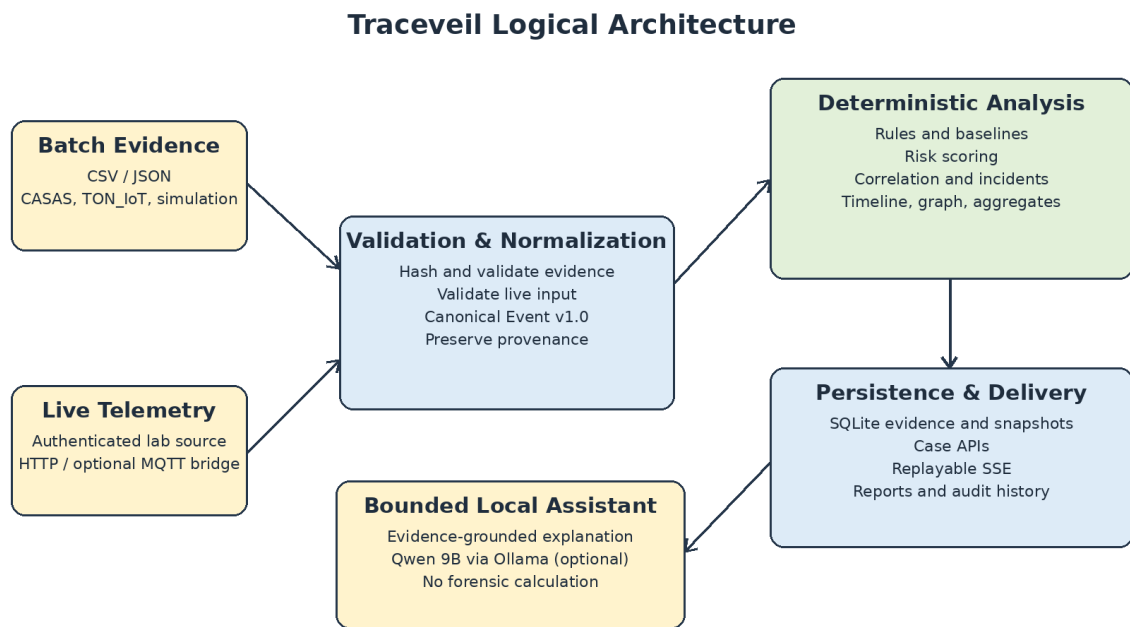


Figure 3.1 Traceveil logical architecture.

Batch CSV/JSON evidence and authenticated laboratory telemetry both pass through validation and normalization. For batch files, the validator checks structure, dataset-specific columns and values, computes SHA-256 hashes, and issues validated records. Live input is authenticated, bounded, timestamped by the backend at ingestion, and stored through durable receipts. Only accepted records are mapped to the canonical schema; rejected data is never silently repaired.

Canonical events retain case/event identifiers, observed and ingested timestamps, event type, entities, network fields where available, source label, attributes, and provenance including evidence ID, source hash, raw-record hash, source reference, adapter version, and normalization version. Observed time and ingestion time remain separate. This provenance is the link between derived findings and original evidence.

The AnalysisService then applies stable filtering and ordering, versioned rules and behavioural baselines, temporal/entity correlation, bounded risk scoring, incident grouping, timeline construction, graph generation, and chart-ready aggregates. Alerts are created only when a qualifying finding is triggered by live evidence; historical batch findings remain findings without becoming live alerts.

3.2 Hardware Description

The hardware path is intended for a controlled laboratory demonstration and does not change the backend evidence contract. The reference node is an ESP32-WROOM-32 development kit connected to simple household-style sensors and an actuator. HTTP over TLS is the default transport; an optional MQTT bridge can forward the same telemetry contract to the existing FastAPI collector.

Table 3.1 Selected hardware components

Component	Selection	Purpose
Microcontroller	ESP32-WROOM-32	Wi-Fi/TLS telemetry source and sequence generation.
Environment	DHT22 on GPIO 4	Temperature and humidity telemetry.
Motion	HC-SR501 PIR on GPIO 27	Household motion/device-state events.
Actuation	5 V relay + sense input	Controlled command and independently observed state.
Operator/status	GPIO 33 button + GPIO 2 LED	Authorized local command path and device status.

The laboratory network is isolated, uses controlled time synchronization, and provisions per-source credentials. The ESP32 maintains a monotonic sequence in non-volatile storage. When MQTT is selected, the bridge uses a durable SQLite spool and forwards accepted frames to the same authenticated HTTP endpoint; direct HTTPS remains the simplest demonstration path.

3.3 Software Description

Table 3.2 Software technology stack

Layer	Technology	Role
Frontend	React, Vite, TypeScript	Case workspace, evidence import, findings, timelines,

Layer	Technology	Role
		graphs, live/reports UI.
Backend	Python, FastAPI, Pydantic	Validation, normalization, analysis, case/live/report APIs.
Analysis	Pandas, NetworkX	Aggregates, correlation, incidents, graph and timeline structures.
Persistence	SQLite	Evidence metadata, canonical events, snapshots, receipts and audit history.
Local AI	Ollama + Qwen 9B	Optional evidence-grounded explanation; no forensic calculation.
Reporting	ReportLab, SMTP	Deterministic PDF generation and approval-gated delivery.

For batch processing, an investigator creates a case, selects a supported source profile, uploads CSV/JSON evidence, reviews validation results, and commits accepted rows for normalization and analysis. For live processing, a live session allow-lists source IDs; authenticated JSON telemetry is accepted at `/api/v1/live/telemetry`, persisted as a receipt, normalized, analysed, and made available to the live interface through replayable SSE. The bounded assistant constructs context only from persisted case records and validates model citations and numerical claims before storing a response.

3.4 Flowchart and Core Algorithms

UML Activity Model Evidence Import and Analysis

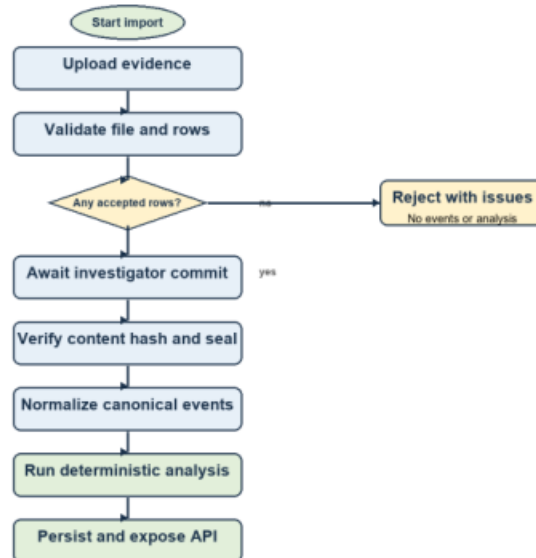


Figure 3.2 Evidence import and deterministic analysis activity model.

3.4.1 Evidence Processing Algorithm

1. Receive case ID, source profile, filename, and original evidence bytes.
2. Validate file type, size, encoding, structure, required columns, and source-specific values; compute the original SHA-256 hash.

3. For each accepted row, create a stable row hash and authorization seal; rejected rows are excluded from normalization.
4. After investigator commit, verify the seal and map each accepted record through its versioned source adapter to Canonical Event v1.0.
5. Persist canonical events, execute deterministic analysis, persist the analysis snapshot, and expose queryable artifacts through case-scoped APIs.

3.4.2 Detection, Risk and Correlation

Table 3.3 Deterministic detection rules

Rule	Condition	Default result
LABEL-001	Configured malicious source label / attack class.	Medium/High/Critical by class; confidence 95.
RATE-001	requests $\geq 3 \times$ positive baseline.	High; confidence 90.
AUTH-001	10 authentication failures for same target/actor within 60 observed seconds.	Critical; confidence 98.
BASELINE-001	Metric exceeds prior mean + max(3 sigma, minimum delta).	High; confidence 80.

Risk is an integer 0-100 derived from severity (30%), confidence (25%), repetition (20%), device criticality (15%), and corroboration (10%). It is a deterministic triage score, not proof of compromise or causation. Correlation requires observed timestamps within the configured window and at least one shared declared entity or network identity. Live sequence controls distinguish exact replay, sequence collision, and regression so duplicates or suspicious sequence reuse do not silently become new evidence.

3.5 Current Implementation Status and Future Extensions

The completion report dated 6 September 2026 records 93 of 143 roadmap sub-items as implemented and verified, corresponding to 65.0% formal roadmap completion. It marks initialization, frontend foundation, evidence validation, canonical normalization, Live IoT Integration, deterministic analytics, and dashboard/investigation APIs as complete, with frontend tests, backend/API/persistence tests, TypeScript/lint checks, production build, and the batch usability showcase passing. The same report keeps real-time delivery acceptance, AI/visualization milestones, reporting automation, and the final physical demonstration as remaining roadmap focus areas, even where software support already exists in the repository.

The reviewed contribution record contains 50 commits through the same date. Atharva Rangnekar is the project lead responsible for planning, repository foundation, frontend

architecture, integration and documentation (34 commits); Aarya Rao owns evidence validation, canonical normalization, deterministic analytics and investigation APIs (10 commits); and Ayra Jha owns Live IoT Integration and its integration/CI fixes (6 commits). The next review stage should emphasize evidence-backed acceptance of the physical demo, representative live scenarios, end-to-end usability, and final documentation rather than expanding scope without validation.

Chapter 3 Summary

Chapter 3 described a common batch/live methodology built around validation, provenance, Canonical Event v1.0, deterministic analysis, local persistence, and investigator-facing delivery. The implementation has progressed beyond the core batch workflow into live IoT integration, while the remaining work is primarily final acceptance, demonstration, and review-stage completion.

References

- [1] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, “TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems,” *IEEE Access*, vol. 8, 2020.
- [2] Washington State University, “CASAS Smart Home Datasets,” CASAS Project, dataset portal: casas.wsu.edu/datasets.
- [3] E. C. P. Neto et al., “CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment,” *Sensors*, vol. 23, no. 13, 2023.
- [4] H. Mohamed, N. Koroniotis, F. Schiliro, and N. Moustafa, “IoT-CAD: A Comprehensive Digital Forensics Dataset for AI-based Cyberattack Attribution Detection Methods in IoT Environments,” *Ad Hoc Networks*, 2025.
- [5] D.-H. Shin et al., “Research on Digital Forensics Analyzing Heterogeneous Internet of Things Incident Investigations,” *Applied Sciences*, vol. 14, no. 3, 2024.
- [6] “Smart Home IoT Forensics: Current Status, Challenges, and Future Directions,” *International Conference on Advancement in Computation & Computer Technologies (InCACCT)*, 2023.
- [7] F. Abdel-Fattah et al., “A Survey of Internet of Things (IoT) Forensics Frameworks and Challenges,” 2023.
- [8] “Device Forensics in Smart Homes: Insights on Advances, Challenges and Future Directions,” *Transactions on Large-Scale Data- and Knowledge-Centered Systems*, Springer, 2024.
- [9] “Cybersecurity and Cyber Forensics for Smart Cities: A Comprehensive Literature Review and Survey,” 2023.
- [10] S. Kim, G. Lee, J. Song, I. Lee, and T. Shon, “A Common Architecture-Based Smart Home Tools and Applications Forensics for Scalable Investigations,” *Computers, Materials & Continua*, vol. 83, no. 1, pp. 661-683, 2025.
- [11] “Large Language Models in Digital Forensics: Capabilities, Challenges and Future Directions,” *Forensic Science International: Digital Investigation*, 2026.
- [12] “ForensicLLM: A Local Large Language Model for Digital Forensics,” *Forensic Science International: Digital Investigation*, 2025.
- [13] National Institute of Standards and Technology, NIST SP 800-86, “Guide to Integrating Forensic Techniques into Incident Response.”
- [14] National Institute of Standards and Technology, NIST SP 800-92, “Guide to Computer Security Log Management.”